

Phizura - Speaking Script

20-minute talk - IWST 2026. Roughly 50 seconds per slide; pace cues in italics. ~2,700 words.

Slide 1 - Title

[0:00 - 0:40]

Good morning, everyone. My name is [your name], and I'm going to talk about Phizura, a tool for recording live coding music performances. The work was done with my co-authors at Inria and the University of Lille.

The one-line version: Phizura records a live coding session not as audio, but as code - so the whole performance can be replayed, edited, and remixed afterwards. And it does this without modifying a single line of the music library it records.

Slide 2 - What is live coding?

[0:40 - 1:50]

Let me start with what live coding actually is, in case it's new to anyone here.

Live coding is a performance practice. An artist sits in front of an audience and writes and modifies source code in real time, and that code generates the output as it runs - usually music, sometimes visuals or even poetry. It's improvised. The code is the score, and the computer performs it while the audience literally watches it being written.

This isn't a fringe activity. It has a serious academic home: the International Conference on Live Coding is now in its tenth year. And there's a whole family of languages built for it - Tidal Cycles, Sonic Pi, Orca, and others.

Slide 3 - Coypu

[1:50 - 3:00]

Our work lives in the Pharo world. The live coding library here is called Coypu - it's a library and a small DSL for making music on the fly, and as far as we know it's the first to bring live coding into Smalltalk.

You write and evaluate code in a Pharo Playground. Coypu turns your expressions into musical sequences and sends them to an audio server - that can be an internal DSP engine called Phausto, or an external one over OSC or MIDI, like SuperCollider.

The key thing to hold onto is this: every evaluation hits the running sound immediately. The performance simply is the sequence of what you evaluate, when you evaluate it, and in what order. Keep that sentence in mind, because that is exactly what we want to record.

Slide 4 - The problem

[3:00 - 4:20]

So here's the problem. How do you capture a live coding performance?

The obvious answer is: record the audio. But audio only captures the result. It throws away the entire compositional process - the edits, the evaluations, and the timing between them. Audio files are also big, awkward to share, and you can never re-run them in a different setup.

The alternative is to capture the compositional data itself. If you have that, you can replay the performance exactly - and even swap in different sounds. It's lightweight text, so it's easy to share and to analyse. This isn't a need we invented: the Tidal Cycles community explicitly asked for a tool that stores every evaluation with its timestamp and supports replay. That thread ended without a solution. No integrated tool existed.

Slide 5 - What Phizura sets out to do

[4:20 - 5:30]

That gap is what Phizura fills. The goals are straightforward.

Record the actual writing and evaluation of expressions, not just the audio. Capture both code executions and keystrokes, each with a precise timestamp. And then emit the session as self-contained, executable Pharo code that replays with the original timing.

Because the replay is code, not video, it stays alive - you can modify it, play it with other sounds, remix it. To our knowledge this is the first tool to record live coding through reflective instrumentation, and the first to produce replay scripts at the granularity of individual evaluations.

Slide 6 - Section: the first attempt

[5:30 - 5:40]

Before I show you how it works, it helps to see how we did it the first time - and why we threw that version away.

Slide 7 - The Announcer approach

[5:40 - 6:50]

The first recorder for Coypu was built during an internship and shown at ESUG 2024. It used Pharo's Announcer and Announcement classes - which are just the observer pattern.

The idea was simple. We went into the Coypu codebase and, at every point where an evaluation could make a sound, we fired an announcement. Then a recorder object subscribed to those announcements and logged each one with a timestamp.

And it worked. But it came with real trade-offs.

Slide 8 - Why it hurt

[6:50 - 8:10]

The recording logic ended up scattered all through the Coypu source. Every time we wanted to record a new kind of event, we had to edit Coypu twice - define a new announcement class, and then insert the firing at the right spot in the pipeline.

That created tight coupling: the recorder depended on announcement classes that lived inside the library, so the two couldn't evolve independently. Wanting to capture keystrokes, or cursor position, or events from other tools would have meant even more invasive surgery.

And the most fragile part: any change to the evaluation pipeline could silently bypass or break the recording hooks, with no warning. All of this pushed us toward a redesign where the recording lives completely outside Coypu.

Slide 9 - Method Proxies

[8:10 - 9:40]

That redesign is built on a library called Method Proxies. This is the tool that makes everything else possible, so let me spend a moment on it.

Method Proxies is a per-method instrumentation library for Pharo. The paradigm is called message-passing control: you can run your own code before a method, after it, or instead of it - and even during the stack unwind when an exception is thrown. Crucially, it leaves the source code of the instrumented method completely intact.

The API is small: you subclass a handler class, override a couple of hook methods, and the framework handles the machinery. Under the hood each method is swapped for a precompiled trap, so it's still JIT-compiled and fast. It's meta-safe, it's unwind-safe, you can install and uninstall at runtime, and the overhead averages about one and a half times. Originally this was a tool for profiling and debugging - we're using it for something creative.

Slide 10 - Comparison

[9:40 - 10:50]

So here's the before-and-after, point by point.

Coypu modifications: the Announcer version needed edits at every recordable site; with Method Proxies, none at all. Coupling: previously tight through shared announcement types; now fully external. Adding a new event: before, edit Coypu and define an announcement; now, you just intercept the relevant method.

Removing the feature: before, you had to revert your Coypu changes; now you just uninstall the proxies and the image is back to original. Keystrokes weren't supported before - now they are. And code emission didn't exist before - now there are two modes, which I'll show you shortly.

Slide 11 - Section: architecture

[10:50 - 11:00]

Let me now walk through how Phizura is actually built.

Slide 12 - Three packages

[11:00 - 11:50]

The system is three packages. The core Phizura package holds the recorder, the instrumentor, and the code emitter. Phizura-Playground is a custom Playground that hosts a recording session and puts the controls in its toolbar. And Phizura-Tests is the test suite.

Functionally there are three jobs: instrument the Playground, collect the events into a timeline, and expose the workflow to the performer. I'll take them in that order.

Slide 13 - Instrumentation

[11:50 - 12:50]

First, instrumentation. A single class, the evaluation instrumentor, decides which methods to intercept and installs a method proxy on each one.

There are two methods that get invoked whenever code is evaluated, and we instrument both. Between them they cover every endpoint of evaluation, so nothing slips through - every evaluation is captured. Both proxies share one handler instance, so all the recording logic lives in one place.

And the headline point: no Coypu class is modified, subclassed, or even mentioned. The instrumentation is entirely external.

Slide 14 - The handler

[12:50 - 14:00]

This is the only piece of code we actually had to write - the handler. It's a subclass of MpHandler with a single method, the before-execution hook.

When an instrumented evaluation method is about to run, this hook fires. The first argument it receives is the source string that's about to be evaluated. So we grab that string, stamp it with the current time, read the cursor position from the Playground page, and push the whole thing onto the recorder as a record entry.

That's it. No Coypu code is touched, and when you uninstall the proxies the running image is left exactly as it was.

Slide 15 - Recording

[14:00 - 14:50]

The recorder keeps two kinds of entries in one chronological timeline. One entry type for each evaluated expression - those come through the method proxies we just saw. And one entry type for each keystroke - those are captured separately, since Pharo already

gives us a keystroke callback to hook into.

The recorder itself owns the list of entries, holds a reference to the instrumentor, and toggles a recording flag. Starting a session installs the proxies; stopping a session uninstalls them and clears the flag - and again, that leaves the image clean.

Slide 16 - The Phizura Playground

[14:50 - 15:50]

Now, why a custom Playground? Because the standard one gives Method Proxies nothing stable to grab onto - the selected text flows through several layers of UI machinery without ever passing through a single method we could reliably hook.

So we wrote a thin subclass that exposes one method which receives the selected text directly - exactly the shape our handler expects. On top of that, the Playground adds five toolbar commands: one to toggle recording, one to reset, one to inspect the current session, and two to emit the replay scripts - Open Records and Open MoofLOD - which brings me to the interesting part.

Slide 17 - Code emission

[15:50 - 16:50]

At the end of a session you have a time-ordered list of timestamped entries. Replay turns that list back into runnable code.

A code emitter walks the entries in order. It subtracts each pair of consecutive timestamps to recover the original delays, and emits Pharo source with 'seconds wait' statements in between. The whole thing is wrapped in a forked block, so the waits don't freeze the UI and you can launch the replay from the same Playground that captured it.

The choice of output format is done with double dispatch between the emitter and the entries, which means you can add new output formats, or new kinds of entry, without disturbing the other side.

Slide 18 - Mode 1: performance code

[16:50 - 17:40]

There are two emission modes. The first, the performance code, is the clean one. Keystrokes are filtered out and only the evaluations remain, separated by the original delays.

What you see here is real emitted output: create an instrument, wait a second, play 128 bars, wait, add a kick pattern, wait, add a rumba on the conga. It's completely self-contained - any image with the same Coypu setup can run this and reproduce the performance verbatim.

Slide 19 - Mode 2: MoofLOD

[17:40 - 18:30]

The second mode is called MoofLOD. It produces the same music, but it also emits an OSC-compatible signal for every single thing that happened - each evaluated expression, each keystroke, each cursor move.

So as the script replays, it doesn't just make sound; it broadcasts the whole session to any OSC-capable consumer. That opens the door to live visualisations, second-screen overlays, or syncing the performance with other tools.

Slide 20 - Related work

[18:30 - 19:10]

Briefly, the neighbours. The Tidal Cycles forum thread laid out the requirements but ended with no tool. The closest real system is Live Writing, from 2015, which records keystroke-level typing in a browser via editor hooks - whereas we use reflective instrumentation and touch neither the editor nor the library.

And there's SHARP, which visualises the version history of code blocks but doesn't capture timing or produce replay scripts. So SHARP and Phizura are complementary - one preserves the structure, the other preserves the temporal performance.

Slide 21 - Generality

[19:10 - 19:40]

One nice consequence: because we instrument all Playground evaluations, not Coypu-specific code, the tool is generic. Visual live coding with Bloc or Roassal could be recorded with no changes at all. It works as a general prototyping recorder - capturing an exploration with its order and pace. And it's useful for teaching: an instructor's session becomes a tiny text file that students can replay, pause, and edit - it's live code, not a video.

Slide 22 - Limitations and future work

[19:40 - 20:00]

Honest limitations: we record evaluated expressions, not the typing and deleting that happens before you hit evaluate, and not mouse-driven actions. And right now it's one session at a time. Future work is a public API for custom capture handlers, and support for multiple Playgrounds so we can record collaborative sessions.

Slide 23-24 - Conclusion and thanks

[wrap]

To wrap up: Phizura is a non-intrusive recorder for live coding in Pharo. Method Proxies let us instrument the Playground's evaluation methods without modifying Coypu at all, and we emit timestamped expressions as executable code that replays with faithful timing. Compared to the announcer version, it's modular, decoupled, and extensible.

And the broader takeaway is that Method Proxies, a tool built for profiling and debugging, turns out to work just as well for creative coding. Thank you - I'm happy to take questions.